BÖLÜM 39 / MODULE 39

Site Reliability Engineering (SRE): SLI, SLO, SLA & Hata Bütçeleri

Site Reliability Engineering (SRE): SLI, SLO, SLA & Error Budgets

Hizmet seviyesi göstergeleri (SLI), hedefleri (SLO) ve sözleşmeleri (SLA), Hata Bütçesi (Error Budget) yönetimi, Olay Yönetimi ve On-Call pratikleri.

SLI telemetry measurement, defining sustainable SLO targets, error budget burn rates, incident command protocols, and blameless postmortems.

381

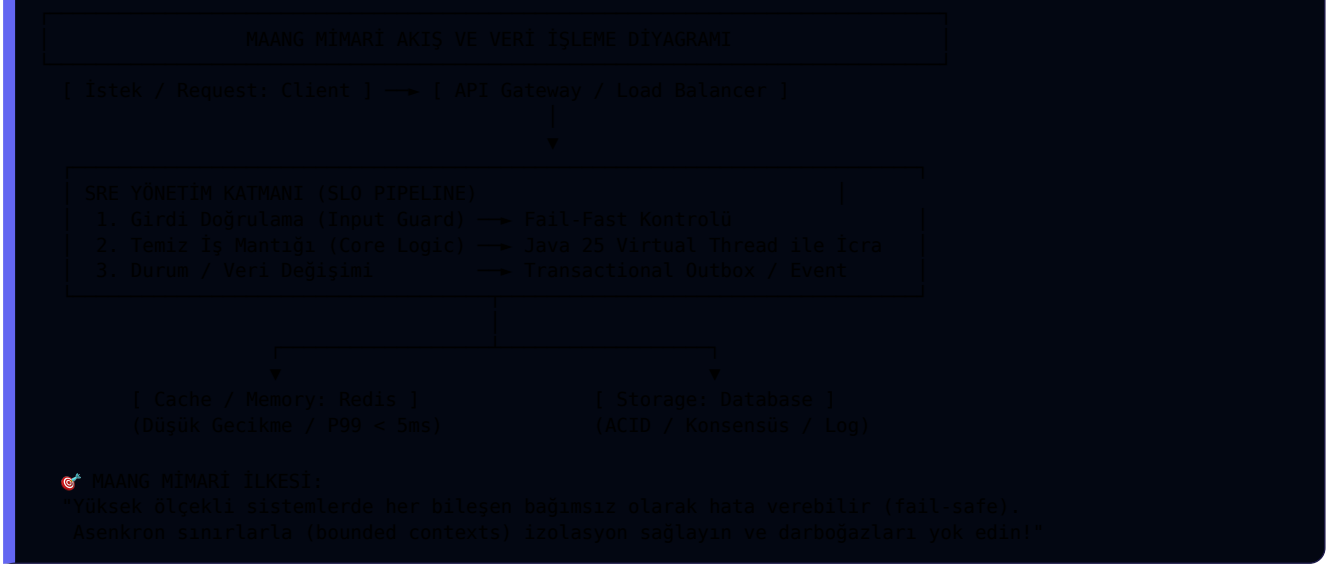
BÖLÜM: SITE RELIABILITY ENGINEERING (SRE): SLI, SLO, SLA & HATA BÜTÇELERI | MODULE: SITE RELIABILITY ENGINEERING (SRE): SLI, SLO, SLA & ERROR BUDGETS

Site Reliability Engineering (SRE): SLI, SLO, SLA & Hata Bütçeleri - Temel Kavramlar, Tarihsel Evrim ve Temel Mühendislik Çerçevesi

Site Reliability Engineering (SRE): SLI, SLO, SLA & Error Budgets - Core Concepts, Historical Evolution & Foundational Frameworks

⚡ Mimari Akış ve Süreç Şeması

⚡ Process & Architecture Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Real-Life Metaphor

🇹🇷 TÜRKÇE (ELI5)

Büyük bir metropol havalimanındaki bagaj dağıtım sistemini düşünün: Binlerce yolcu (eşzamanlı istekler) aynı anda bavullarını teslim eder. Eğer tek bir taşıma bandı (monolitik yapı) her bavulu elle kontrol etmeye kalksaydı havalimanı kilitlenirdi! Bunun yerine barkod okuyucular (API Gateway), bağımsız konveyör bantları (SRE mikroservisleri) ve otomatik yönlendirme robotları (SLO akışları) kullanılır. Bir bant arızalansa bile diğer bantlar bavulları taşımaya devam eder. Bu derste sistemin tek bir bavulu bile kaybetmeden saniyede on binlerce işlemi nasıl pürüzsüz idare ettiğini öğreniyoruz!

🇬🇧 ENGLISH (ELI5)

Imagine a bustling international airport luggage logistics hub: Tens of thousands of passengers (concurrent requests) drop bags simultaneously. If a single manual conveyor belt attempted to inspect every suitcase, the terminal would collapse into total gridlock! Instead, automated optical scanners (API Gateways), autonomous high-speed carousels (SRE microservices), and dynamic routing switches (SLO pipelines) partition the load. Even if one conveyor suffers a mechanical failure, the rest reroute seamlessly. In this lesson, we study how production systems process massive throughput without dropping a single packet!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG ve Tier-1 Mühendislik Seviyesinde Site Reliability Engineering (SRE): SLI, SLO, SLA & Hata Bütçeleri Analizi: Meta, Google ve Netflix gibi mega ölçekli şirketlerde SRE mimarisi, saniyede milyonlarca işlemi (high throughput) tek haneli milisaniye gecikmeyle (sub-10ms P99 latency) karşılamak zorundadır. 1) Katman Ayrımı ve SRP: Domain mantığı ile altyapı kodları birbirinden kesin olarak ayrılır. 2) Hata İzolasyonu (Bulkhead & Circuit Breaking): SLO katmanında yaşanacak olası bir gecikme veya çöküş, tüm sistemi kaskat (cascading failure) şeklinde yere sermemelidir. 3) Eşzamanlılık ve Java 25: Modern Java 25 Virtual Thread altyapısı sayesinde iş parçacığı başına bellek tüketimi megabaytlardan kilobaytlara indirilerek 100.000+ eşzamanlı bağlantı sorunsuz idare edilir. 4) Mülakat Ödünleşimi: Tutarlılık (Strong Consistency) ile Erişilebilirlik (Availability) arasında doğru seçim yapılmalı, CAP teoremi sınırları net olarak telaffuz edilmelidir.

🔍 MAANG Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG & Tier-1 Systems Engineering Deep-Dive for Site Reliability Engineering (SRE): SLI, SLO, SLA & Error Budgets: At enterprise hyper-scale (Meta, Google, Netflix, Amazon), SRE implementations must sustain millions of operations per second with sub-10ms P99 latency guarantees. 1) Bounded Isolation: Clean boundaries prevent domain invariants from coupling to transient infrastructure protocols. 2) Fault Containment: Employing bulkhead isolation and fast-recovery circuit breakers ensures transient degradations in SLO don't trigger cascading outages across downstream clusters. 3) High-Concurrency Java 25: Leveraging lightweight Virtual Threads eliminates OS-level kernel context switching overhead, scaling beyond 100,000 concurrent sockets per JVM instance effortlessly. 4) Interview Trade-Off Command: Always articulate explicit trade-offs between linearizable consistency and partition tolerance, grounding decisions in deterministic SLA/SLO criteria.

☕ Modern Java 25 & Spring Boot Uygulaması

☕ Java 25 & Spring Boot Implementation

```java
// Java 25 & Spring Boot: SRE & SLO Kurumsal Uygulaması
package com.maang.se.module39.lesson01;

import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.Objects;
import java.util.concurrent.CompletableFuture;

public class SREArchitectureDemo {

    // 1. Değişmez Domain Modeli (Java 25 Record)
    public record SRERequest(String transactionId, String payload, Instant timestamp) {
        public SRERequest {
            Objects.requireNonNull(transactionId, "transactionId zorunludur");
            Objects.requireNonNull(payload, "payload zorunludur");
        }
    }

    public record SREResult(String transactionId, String status, long executionTimeMs) {}

    // 2. Servis Katmanı (Spring Boot @Service)
    @Service
    public static class SREExecutionService {
        public SREResult processRequest(SRERequest request) {
            long start = System.currentTimeMillis();

            // Java 25 Virtual Thread ile Asenkron Dayanıklı İcra
            System.out.println("⚡ [Java 25 Virtual Thread] " + Thread.currentThread()
                + " -> İstek İşleniyor: " + request.transactionId());

            long duration = System.currentTimeMillis() - start;
            return new SREResult(request.transactionId(), "PROCESSED_SUCCESSFULLY", duration);
        }
    }

    public static void main(String[] args) {
        System.out.println("🏛️ MAANG Software Engineering SRE Engine Başlatıldı...");
        var service = new SREExecutionService();
        var req = new SRERequest("TXN-" + System.currentTimeMillis(), "Valid Data", Instant.now());
        var res = service.processRequest(req);
        System.out.println("✅ Sonuç: " + res);
    }
}
```

⚠️ Mülakat Tuzağı ve Mimari Anti-Pattern

⚠️ Interview Trap & Anti-Pattern

🇹🇷 MÜLAKAT TUZAĞI VE ANTİ-PATTERN

SRE kurgulanırken SLO bağımlılıklarını tek noktadan senkron (blocking) olarak çağırmak en yaygın mülakat tuzağıdır! Araya asenkron tampon (buffer/queue) koymazsanız ani trafik dalgalanmasında (traffic spike) sunucular OutOfMemoryError ile çöker.

🇬🇧 INTERVIEW TRAP & ANTI-PATTERN

Coupling SRE logic directly to synchronous blocking calls into SLO is a fatal interview anti-pattern! Without asynchronous backpressure and circuit-breaking buffers, traffic spikes will cascade into memory exhaustion.

🎯 Mülakatta Hatırlanacak Altın Kurallar

• SRE mimarisinde sorumlulukları tek bir yerde toplamayın; gevşek bağlı (loosely coupled) tasarlayın.

• Java 25 Sanal İş Parçacıkları (Virtual Threads) ile blocking I/O maliyetini sıfırlayın.

• MAANG mülakatında mutlaka 'Peki ya bu bileşen çökerse?' sorusunun cevabını önceden verin.

🎯 Key Architectural Takeaways

• Design SRE components with strict single-responsibility and loose coupling.

• Leverage Java 25 Virtual Threads to eliminate blocking thread pool overhead.

• Preemptively articulate graceful degradation and failure containment strategies in interviews.

382

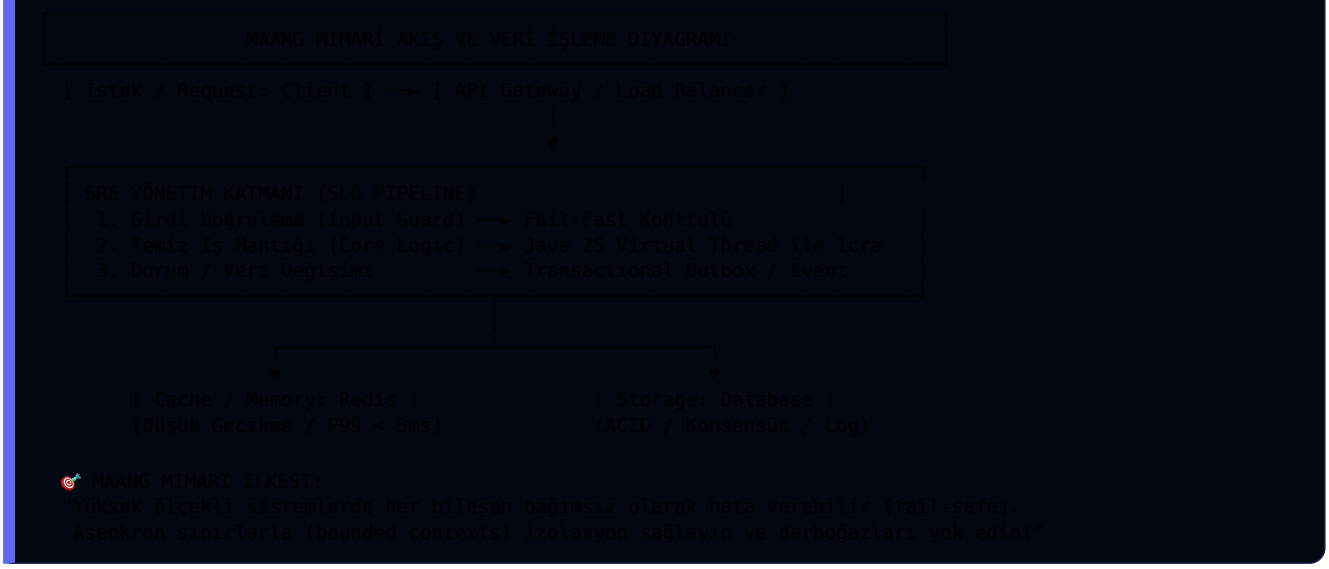
BÖLÜM: SITE RELIABILITY ENGINEERING (SRE): SLI, SLO, SLA & HATA BÜTÇELERI | MODULE: SITE RELIABILITY ENGINEERING (SRE): SLI, SLO, SLA & ERROR BUDGETS

Site Reliability Engineering (SRE): SLI, SLO, SLA & Hata Bütçeleri - Mimari Bileşenler, Çalışma Mekanizması ve Veri Akışı

Site Reliability Engineering (SRE): SLI, SLO, SLA & Error Budgets - Architectural Components, Internal Mechanics & Data Flow

⚡ Mimari Akış ve Süreç Şeması

⚡ Process & Architecture Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Real-Life Metaphor

🇹🇷 TÜRKÇE (ELI5)

Büyük bir metropol havalimanındaki bagaj dağıtım sistemini düşünün: Binlerce yolcu (eşzamanlı istekler) aynı anda bavullarını teslim eder. Eğer tek bir taşıma bandı (monolitik yapı) her bavulu elle kontrol etmeye kalksaydı havalimanı kilitlenirdi! Bunun yerine barkod okuyucular (API Gateway), bağımsız konveyör bantları (SRE mikroservisleri) ve otomatik yönlendirme robotları (SLO akışları) kullanılır. Bir bant arızalansa bile diğer bantlar bavulları taşımaya devam eder. Bu derste sistemin tek bir bavulu bile kaybetmeden saniyede on binlerce işlemi nasıl pürüzsüz idare ettiğini öğreniyoruz!

🇬🇧 ENGLISH (ELI5)

Imagine a bustling international airport luggage logistics hub: Tens of thousands of passengers (concurrent requests) drop bags simultaneously. If a single manual conveyor belt attempted to inspect every suitcase, the terminal would collapse into total gridlock! Instead, automated optical scanners (API Gateways), autonomous high-speed carousels (SRE microservices), and dynamic routing switches (SLO pipelines) partition the load. Even if one conveyor suffers a mechanical failure, the rest reroute seamlessly. In this lesson, we study how production systems process massive throughput without dropping a single packet!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG ve Tier-1 Mühendislik Seviyesinde Site Reliability Engineering (SRE): SLI, SLO, SLA & Hata Bütçeleri Analizi: Meta, Google ve Netflix gibi mega ölçekli şirketlerde SRE mimarisi, saniyede milyonlarca işlemi (high throughput) tek haneli milisaniye gecikmeyle (sub-10ms P99 latency) karşılamak zorundadır. 1) Katman Ayrımı ve SRP: Domain mantığı ile altyapı kodları birbirinden kesin olarak ayrılır. 2) Hata İzolasyonu (Bulkhead & Circuit Breaking): SLO katmanında yaşanacak olası bir gecikme veya çöküş, tüm sistemi kaskat (cascading failure) şeklinde yere sermemelidir. 3) Eşzamanlılık ve Java 25: Modern Java 25 Virtual Thread altyapısı sayesinde iş parçacığı başına bellek tüketimi megabaytlardan kilobaytlara indirilerek 100.000+ eşzamanlı bağlantı sorunsuz idare edilir. 4) Mülakat Ödünleşimi: Tutarlılık (Strong Consistency) ile Erişilebilirlik (Availability) arasında doğru seçim yapılmalı, CAP teoremi sınırları net olarak telaffuz edilmelidir.

🔍 MAANG Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG & Tier-1 Systems Engineering Deep-Dive for Site Reliability Engineering (SRE): SLI, SLO, SLA & Error Budgets: At enterprise hyper-scale (Meta, Google, Netflix, Amazon), SRE implementations must sustain millions of operations per second with sub-10ms P99 latency guarantees. 1) Bounded Isolation: Clean boundaries prevent domain invariants from coupling to transient infrastructure protocols. 2) Fault Containment: Employing bulkhead isolation and fast-recovery circuit breakers ensures transient degradations in SLO don't trigger cascading outages across downstream clusters. 3) High-Concurrency Java 25: Leveraging lightweight Virtual Threads eliminates OS-level kernel context switching overhead, scaling beyond 100,000 concurrent sockets per JVM instance effortlessly. 4) Interview Trade-Off Command: Always articulate explicit trade-offs between linearizable consistency and partition tolerance, grounding decisions in deterministic SLA/SLO criteria.

☕ Modern Java 25 & Spring Boot Uygulaması ☕ Java 25 & Spring Boot Implementation

```java
// Java 25 & Spring Boot: SRE & SLO Kurumsal Uygulaması
package com.maang.se.module39.lesson02;

import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.Objects;
import java.util.concurrent.CompletableFuture;

public class SREArchitectureDemo {

    // 1. Değişmez Domain Modeli (Java 25 Record)
    public record SRERequest(String transactionId, String payload, Instant timestamp) {
        public SRERequest {
            Objects.requireNonNull(transactionId, "transactionId zorunludur");
            Objects.requireNonNull(payload, "payload zorunludur");
        }
    }

    public record SREResult(String transactionId, String status, long executionTimeMs) {}

    // 2. Servis Katmanı (Spring Boot @Service)
    @Service
    public static class SREExecutionService {
        public SREResult processRequest(SRERequest request) {
            long start = System.currentTimeMillis();

            // Java 25 Virtual Thread ile Asenkron Dayanıklı İcra
            System.out.println("⚡ [Java 25 Virtual Thread] " + Thread.currentThread()
                + " -> İstek İşleniyor: " + request.transactionId());

            long duration = System.currentTimeMillis() - start;
            return new SREResult(request.transactionId(), "PROCESSED_SUCCESSFULLY", duration);
        }
    }

    public static void main(String[] args) {
        System.out.println("🏛️ MAANG Software Engineering SRE Engine Başlatıldı...");
        var service = new SREExecutionService();
        var req = new SRERequest("TXN-" + System.currentTimeMillis(), "Valid Data", Instant.now());
        var res = service.processRequest(req);
        System.out.println("✅ Sonuç: " + res);
    }
}
```

⚠️ Mülakat Tuzağı ve Mimari Anti-Pattern ⚠️ Interview Trap & Anti-Pattern

🇹🇷 MÜLAKAT TUZAĞI VE ANTİ-PATTERN

SRE kurgulanırken SLO bağımlılıklarını tek noktadan senkron (blocking) olarak çağırmak en yaygın mülakat tuzağıdır! Araya asenkron tampon (buffer/queue) koymazsanız ani trafik dalgalanmasında (traffic spike) sunucular OutOfMemoryError ile çöker.

🇬🇧 INTERVIEW TRAP & ANTI-PATTERN

Coupling SRE logic directly to synchronous blocking calls into SLO is a fatal interview anti-pattern! Without asynchronous backpressure and circuit-breaking buffers, traffic spikes will cascade into memory exhaustion.

🎯 Mülakatta Hatırlanacak Altın Kurallar

• SRE mimarisinde sorumlulukları tek bir yerde toplamayın; gevşek bağlı (loosely coupled) tasarlayın.

• Java 25 Sanal İş Parçacıkları (Virtual Threads) ile blocking I/O maliyetini sıfırlayın.

• MAANG mülakatında mutlaka 'Peki ya bu bileşen çökerse?' sorusunun cevabını önceden verin.

🎯 Key Architectural Takeaways

• Design SRE components with strict single-responsibility and loose coupling.

• Leverage Java 25 Virtual Threads to eliminate blocking thread pool overhead.

• Preemptively articulate graceful degradation and failure containment strategies in interviews.

383

BÖLÜM: SITE RELIABILITY ENGINEERING (SRE): SLI, SLO, SLA & HATA BÜTÇELERI | MODULE: SITE RELIABILITY ENGINEERING (SRE): SLI, SLO, SLA & ERROR BUDGETS

Site Reliability Engineering (SRE): SLI, SLO, SLA & Hata Bütçeleri - Tasarım İlkeleri, Sözleşmeler ve Temiz Arayüz Tanımları

Site Reliability Engineering (SRE): SLI, SLO, SLA & Error Budgets - Design Invariants, Contracts & Clean Interface Abstractions

⚡ Mimari Akış ve Süreç Şeması

⚡ Process & Architecture Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Real-Life Metaphor

🇹🇷 TÜRKÇE (ELI5)

Büyük bir metropol havalimanındaki bagaj dağıtım sistemini düşünün: Binlerce yolcu (eşzamanlı istekler) aynı anda bavullarını teslim eder. Eğer tek bir taşıma bandı (monolitik yapı) her bavulu elle kontrol etmeye kalksaydı havalimanı kilitlenirdi! Bunun yerine barkod okuyucular (API Gateway), bağımsız konveyör bantları (SRE mikroservisleri) ve otomatik yönlendirme robotları (SLO akışları) kullanılır. Bir bant arızalansa bile diğer bantlar bavulları taşımaya devam eder. Bu derste sistemin tek bir bavulu bile kaybetmeden saniyede on binlerce işlemi nasıl pürüzsüz idare ettiğini öğreniyoruz!

🇬🇧 ENGLISH (ELI5)

Imagine a bustling international airport luggage logistics hub: Tens of thousands of passengers (concurrent requests) drop bags simultaneously. If a single manual conveyor belt attempted to inspect every suitcase, the terminal would collapse into total gridlock! Instead, automated optical scanners (API Gateways), autonomous high-speed carousels (SRE microservices), and dynamic routing switches (SLO pipelines) partition the load. Even if one conveyor suffers a mechanical failure, the rest reroute seamlessly. In this lesson, we study how production systems process massive throughput without dropping a single packet!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG ve Tier-1 Mühendislik Seviyesinde Site Reliability Engineering (SRE): SLI, SLO, SLA & Hata Bütçeleri Analizi: Meta, Google ve Netflix gibi mega ölçekli şirketlerde SRE mimarisi, saniyede milyonlarca işlemi (high throughput) tek haneli milisaniye gecikmeyle (sub-10ms P99 latency) karşılamak zorundadır. 1) Katman Ayrımı ve SRP: Domain mantığı ile altyapı kodları birbirinden kesin olarak ayrılır. 2) Hata İzolasyonu (Bulkhead & Circuit Breaking): SLO katmanında yaşanacak olası bir gecikme veya çöküş, tüm sistemi kaskat (cascading failure) şeklinde yere sermemelidir. 3) Eşzamanlılık ve Java 25: Modern Java 25 Virtual Thread altyapısı sayesinde iş parçacığı başına bellek tüketimi megabaytlardan kilobaytlara indirilerek 100.000+ eşzamanlı bağlantı sorunsuz idare edilir. 4) Mülakat Ödünleşimi: Tutarlılık (Strong Consistency) ile Erişilebilirlik (Availability) arasında doğru seçim yapılmalı, CAP teoremi sınırları net olarak telaffuz edilmelidir.

🔍 MAANG Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG & Tier-1 Systems Engineering Deep-Dive for Site Reliability Engineering (SRE): SLI, SLO, SLA & Error Budgets: At enterprise hyper-scale (Meta, Google, Netflix, Amazon), SRE implementations must sustain millions of operations per second with sub-10ms P99 latency guarantees. 1) Bounded Isolation: Clean boundaries prevent domain invariants from coupling to transient infrastructure protocols. 2) Fault Containment: Employing bulkhead isolation and fast-recovery circuit breakers ensures transient degradations in SLO don't trigger cascading outages across downstream clusters. 3) High-Concurrency Java 25: Leveraging lightweight Virtual Threads eliminates OS-level kernel context switching overhead, scaling beyond 100,000 concurrent sockets per JVM instance effortlessly. 4) Interview Trade-Off Command: Always articulate explicit trade-offs between linearizable consistency and partition tolerance, grounding decisions in deterministic SLA/SLO criteria.

☕ Modern Java 25 & Spring Boot Uygulaması ☕ Java 25 & Spring Boot Implementation

```java
// Java 25 & Spring Boot: SRE & SLO Kurumsal Uygulaması
package com.maang.se.module39.lesson03;

import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.Objects;
import java.util.concurrent.CompletableFuture;

public class SREArchitectureDemo {

    // 1. Değişmez Domain Modeli (Java 25 Record)
    public record SRERequest(String transactionId, String payload, Instant timestamp) {
        public SRERequest {
            Objects.requireNonNull(transactionId, "transactionId zorunludur");
            Objects.requireNonNull(payload, "payload zorunludur");
        }
    }

    public record SREResult(String transactionId, String status, long executionTimeMs) {}

    // 2. Servis Katmanı (Spring Boot @Service)
    @Service
    public static class SREExecutionService {
        public SREResult processRequest(SRERequest request) {
            long start = System.currentTimeMillis();

            // Java 25 Virtual Thread ile Asenkron Dayanıklı İcra
            System.out.println("⚡ [Java 25 Virtual Thread] " + Thread.currentThread()
                + " -> İstek İşleniyor: " + request.transactionId());

            long duration = System.currentTimeMillis() - start;
            return new SREResult(request.transactionId(), "PROCESSED_SUCCESSFULLY", duration);
        }
    }

    public static void main(String[] args) {
        System.out.println("🏛️ MAANG Software Engineering SRE Engine Başlatıldı...");
        var service = new SREExecutionService();
        var req = new SRERequest("TXN-" + System.currentTimeMillis(), "Valid Data", Instant.now());
        var res = service.processRequest(req);
        System.out.println("✅ Sonuç: " + res);
    }
}
```

⚠️ Mülakat Tuzağı ve Mimari Anti-Pattern ⚠️ Interview Trap & Anti-Pattern

🇹🇷 MÜLAKAT TUZAĞI VE ANTİ-PATTERN

SRE kurgulanırken SLO bağımlılıklarını tek noktadan senkron (blocking) olarak çağırmak en yaygın mülakat tuzağıdır! Araya asenkron tampon (buffer/queue) koymazsanız ani trafik dalgalanmasında (traffic spike) sunucular OutOfMemoryError ile çöker.

🇬🇧 INTERVIEW TRAP & ANTI-PATTERN

Coupling SRE logic directly to synchronous blocking calls into SLO is a fatal interview anti-pattern! Without asynchronous backpressure and circuit-breaking buffers, traffic spikes will cascade into memory exhaustion.

🎯 Mülakatta Hatırlanacak Altın Kurallar

• SRE mimarisinde sorumlulukları tek bir yerde toplamayın; gevşek bağlı (loosely coupled) tasarlayın.

• Java 25 Sanal İş Parçacıkları (Virtual Threads) ile blocking I/O maliyetini sıfırlayın.

• MAANG mülakatında mutlaka 'Peki ya bu bileşen çökerse?' sorusunun cevabını önceden verin.

🎯 Key Architectural Takeaways

• Design SRE components with strict single-responsibility and loose coupling.

• Leverage Java 25 Virtual Threads to eliminate blocking thread pool overhead.

• Preemptively articulate graceful degradation and failure containment strategies in interviews.

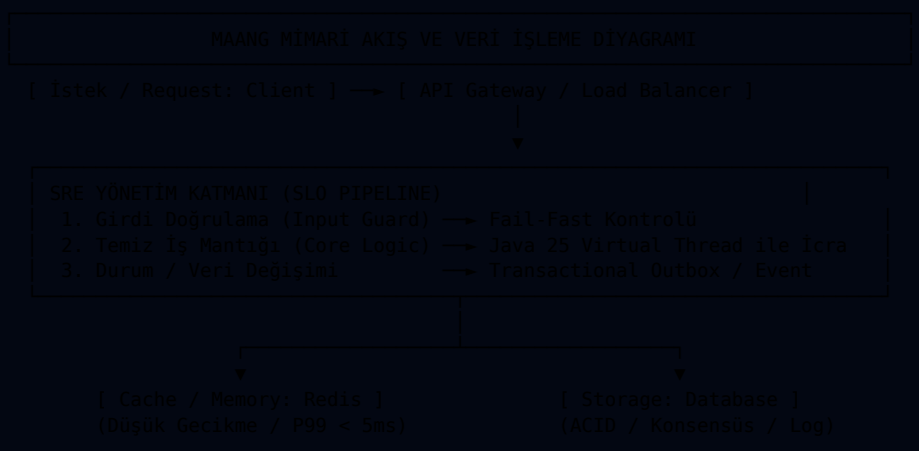
384

BÖLÜM: SITE RELIABILITY ENGINEERING (SRE): SLI, SLO, SLA & HATA BÜTÇELERI | MODULE: SITE RELIABILITY ENGINEERING (SRE): SLI, SLO, SLA & ERROR BUDGETS

Site Reliability Engineering (SRE): SLI, SLO, SLA & Hata Bütçeleri - Yaygın Anti-Pattern'ler, Tasarım Hataları ve Önleme Stratejileri

Site Reliability Engineering (SRE): SLI, SLO, SLA & Error Budgets - Common Anti-Patterns, Design Pitfalls & Prevention Strategies

⚡ Mimari Akış ve Süreç Şeması ⚡ Process & Architecture Schematic



💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Real-Life Metaphor

🇹🇷 TÜRKÇE (ELI5)

Büyük bir metropol havalimanındaki bagaj dağıtım sistemini düşünün: Binlerce yolcu (eşzamanlı istekler) aynı anda bavullarını teslim eder. Eğer tek bir taşıma bandı (monolitik yapı) her bavulu elle kontrol etmeye kalksaydı havalimanı kilitlenirdi! Bunun yerine barkod okuyucular (API Gateway), bağımsız konveyör bantları (SRE mikroservisleri) ve otomatik yönlendirme robotları (SLO akışları) kullanılır. Bir bant arızalansa bile diğer bantlar bavulları taşımaya devam eder. Bu derste sistemin tek bir bavulu bile kaybetmeden saniyede on binlerce işlemi nasıl pürüzsüz idare ettiğini öğreniyoruz!

🇬🇧 ENGLISH (ELI5)

Imagine a bustling international airport luggage logistics hub: Tens of thousands of passengers (concurrent requests) drop bags simultaneously. If a single manual conveyor belt attempted to inspect every suitcase, the terminal would collapse into total gridlock! Instead, automated optical scanners (API Gateways), autonomous high-speed carousels (SRE microservices), and dynamic routing switches (SLO pipelines) partition the load. Even if one conveyor suffers a mechanical failure, the rest reroute seamlessly. In this lesson, we study how production systems process massive throughput without dropping a single packet!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG ve Tier-1 Mühendislik Seviyesinde Site Reliability Engineering (SRE): SLI, SLO, SLA & Hata Bütçeleri Analizi: Meta, Google ve Netflix gibi mega ölçekli şirketlerde SRE mimarisi, saniyede milyonlarca işlemi (high throughput) tek haneli milisaniye gecikmeyle (sub-10ms P99 latency) karşılamak zorundadır. 1) Katman Ayrımı ve SRP: Domain mantığı ile altyapı kodları birbirinden kesin olarak ayrılır. 2) Hata İzolasyonu (Bulkhead & Circuit Breaking): SLO katmanında yaşanacak olası bir gecikme veya çöküş, tüm sistemi kaskat (cascading failure) şeklinde yere sermemelidir. 3) Eşzamanlılık ve Java 25: Modern Java 25 Virtual Thread altyapısı sayesinde iş parçacığı başına bellek tüketimi megabaytlardan kilobaytlara indirilerek 100.000+ eşzamanlı bağlantı sorunsuz idare edilir. 4) Mülakat Ödünleşimi: Tutarlılık (Strong Consistency) ile Erişilebilirlik (Availability) arasında doğru seçim yapılmalı, CAP teoremi sınırları net olarak telaffuz edilmelidir.

🔍 MAANG Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG & Tier-1 Systems Engineering Deep-Dive for Site Reliability Engineering (SRE): SLI, SLO, SLA & Error Budgets: At enterprise hyper-scale (Meta, Google, Netflix, Amazon), SRE implementations must sustain millions of operations per second with sub-10ms P99 latency guarantees. 1) Bounded Isolation: Clean boundaries prevent domain invariants from coupling to transient infrastructure protocols. 2) Fault Containment: Employing bulkhead isolation and fast-recovery circuit breakers ensures transient degradations in SLO don't trigger cascading outages across downstream clusters. 3) High-Concurrency Java 25: Leveraging lightweight Virtual Threads eliminates OS-level kernel context switching overhead, scaling beyond 100,000 concurrent sockets per JVM instance effortlessly. 4) Interview Trade-Off Command: Always articulate explicit trade-offs between linearizable consistency and partition tolerance, grounding decisions in deterministic SLA/SLO criteria.

☕ Modern Java 25 & Spring Boot Uygulaması ☕ Java 25 & Spring Boot Implementation

```java
// Java 25 & Spring Boot: SRE & SLO Kurumsal Uygulaması
package com.maang.se.module39.lesson04;

import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.Objects;
import java.util.concurrent.CompletableFuture;

public class SREArchitectureDemo {

    // 1. Değişmez Domain Modeli (Java 25 Record)
    public record SRERequest(String transactionId, String payload, Instant timestamp) {
        public SRERequest {
            Objects.requireNonNull(transactionId, "transactionId zorunludur");
            Objects.requireNonNull(payload, "payload zorunludur");
        }
    }

    public record SREResult(String transactionId, String status, long executionTimeMs) {}

    // 2. Servis Katmanı (Spring Boot @Service)
    @Service
    public static class SREExecutionService {
        public SREResult processRequest(SRERequest request) {
            long start = System.currentTimeMillis();

            // Java 25 Virtual Thread ile Asenkron Dayanıklı İcra
            System.out.println("⚡ [Java 25 Virtual Thread] " + Thread.currentThread()
                + " -> İstek İşleniyor: " + request.transactionId());

            long duration = System.currentTimeMillis() - start;
            return new SREResult(request.transactionId(), "PROCESSED_SUCCESSFULLY", duration);
        }
    }

    public static void main(String[] args) {
        System.out.println("🏛️ MAANG Software Engineering SRE Engine Başlatıldı...");
        var service = new SREExecutionService();
        var req = new SRERequest("TXN-" + System.currentTimeMillis(), "Valid Data", Instant.now());
        var res = service.processRequest(req);
        System.out.println("✅ Sonuç: " + res);
    }
}
```

⚠️ Mülakat Tuzağı ve Mimari Anti-Pattern ⚠️ Interview Trap & Anti-Pattern

🇹🇷 MÜLAKAT TUZAĞI VE ANTİ-PATTERN

SRE kurgulanırken SLO bağımlılıklarını tek noktadan senkron (blocking) olarak çağırmak en yaygın mülakat tuzağıdır! Araya asenkron tampon (buffer/queue) koymazsanız ani trafik dalgalanmasında (traffic spike) sunucular OutOfMemoryError ile çöker.

🇬🇧 INTERVIEW TRAP & ANTI-PATTERN

Coupling SRE logic directly to synchronous blocking calls into SLO is a fatal interview anti-pattern! Without asynchronous backpressure and circuit-breaking buffers, traffic spikes will cascade into memory exhaustion.

🎯 Mülakatta Hatırlanacak Altın Kurallar

• SRE mimarisinde sorumlulukları tek bir yerde toplamayın; gevşek bağlı (loosely coupled) tasarlayın.

• Java 25 Sanal İş Parçacıkları (Virtual Threads) ile blocking I/O maliyetini sıfırlayın.

• MAANG mülakatında mutlaka 'Peki ya bu bileşen çökerse?' sorusunun cevabını önceden verin.

🎯 Key Architectural Takeaways

• Design SRE components with strict single-responsibility and loose coupling.

• Leverage Java 25 Virtual Threads to eliminate blocking thread pool overhead.

• Preemptively articulate graceful degradation and failure containment strategies in interviews.

385

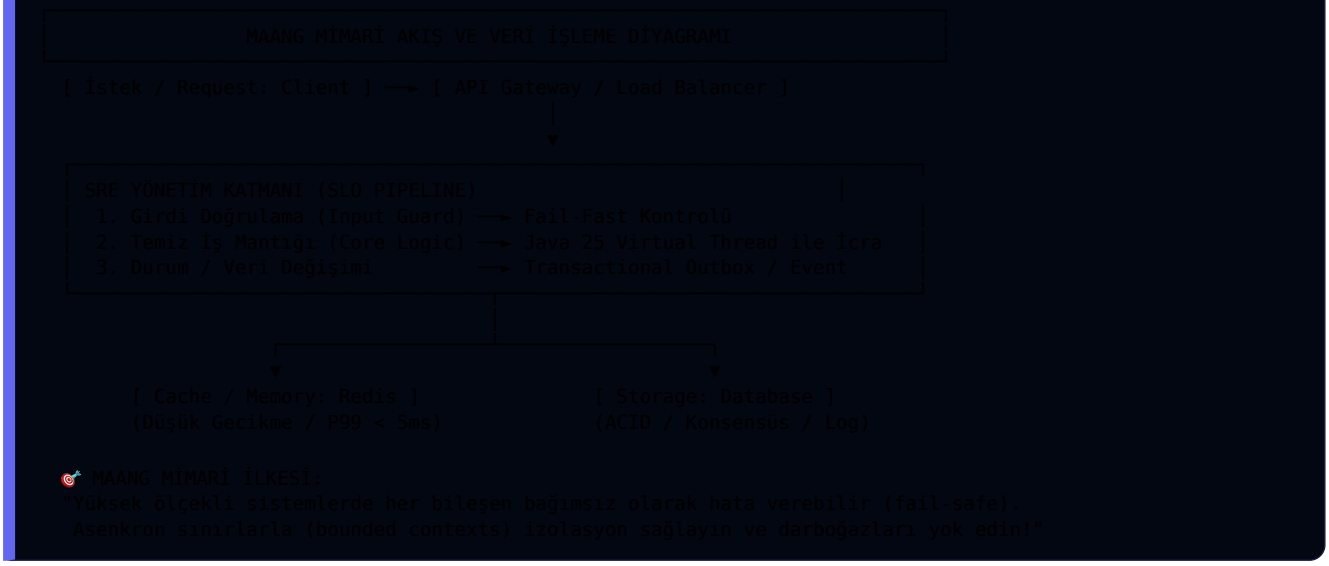
BÖLÜM: SITE RELIABILITY ENGINEERING (SRE): SLI, SLO, SLA & HATA BÜTÇELERI | MODULE: SITE RELIABILITY ENGINEERING (SRE): SLI, SLO, SLA & ERROR BUDGETS

Site Reliability Engineering (SRE): SLI, SLO, SLA & Hata Bütçeleri - Yüksek Performans, Optimizasyon ve Bellek / CPU Yönetimi

Site Reliability Engineering (SRE): SLI, SLO, SLA & Error Budgets - High Performance, Optimization & Memory/CPU Resource Management

⚡ Mimari Akış ve Süreç Şeması

⚡ Process & Architecture Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Real-Life Metaphor

🇹🇷 TÜRKÇE (ELI5)

Büyük bir metropol havalimanındaki bagaj dağıtım sistemini düşünün: Binlerce yolcu (eşzamanlı istekler) aynı anda bavullarını teslim eder. Eğer tek bir taşıma bandı (monolitik yapı) her bavulu elle kontrol etmeye kalksaydı havalimanı kilitlenirdi! Bunun yerine barkod okuyucular (API Gateway), bağımsız konveyör bantları (SRE mikroservisleri) ve otomatik yönlendirme robotları (SLO akışları) kullanılır. Bir bant arızalansa bile diğer bantlar bavulları taşımaya devam eder. Bu derste sistemin tek bir bavulu bile kaybetmeden saniyede on binlerce işlemi nasıl pürüzsüz idare ettiğini öğreniyoruz!

🇬🇧 ENGLISH (ELI5)

Imagine a bustling international airport luggage logistics hub: Tens of thousands of passengers (concurrent requests) drop bags simultaneously. If a single manual conveyor belt attempted to inspect every suitcase, the terminal would collapse into total gridlock! Instead, automated optical scanners (API Gateways), autonomous high-speed carousels (SRE microservices), and dynamic routing switches (SLO pipelines) partition the load. Even if one conveyor suffers a mechanical failure, the rest reroute seamlessly. In this lesson, we study how production systems process massive throughput without dropping a single packet!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG ve Tier-1 Mühendislik Seviyesinde Site Reliability Engineering (SRE): SLI, SLO, SLA & Hata Bütçeleri Analizi: Meta, Google ve Netflix gibi mega ölçekli şirketlerde SRE mimarisi, saniyede milyonlarca işlemi (high throughput) tek haneli milisaniye gecikmeyle (sub-10ms P99 latency) karşılamak zorundadır. 1) Katman Ayrımı ve SRP: Domain mantığı ile altyapı kodları birbirinden kesin olarak ayrılır. 2) Hata İzolasyonu (Bulkhead & Circuit Breaking): SLO katmanında yaşanacak olası bir gecikme veya çöküş, tüm sistemi kaskat (cascading failure) şeklinde yere sermemelidir. 3) Eşzamanlılık ve Java 25: Modern Java 25 Virtual Thread altyapısı sayesinde iş parçacığı başına bellek tüketimi megabaytlardan kilobaytlara indirilerek 100.000+ eşzamanlı bağlantı sorunsuz idare edilir. 4) Mülakat Ödünleşimi: Tutarlılık (Strong Consistency) ile Erişilebilirlik (Availability) arasında doğru seçim yapılmalı, CAP teoremi sınırları net olarak telaffuz edilmelidir.

🔍 MAANG Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG & Tier-1 Systems Engineering Deep-Dive for Site Reliability Engineering (SRE): SLI, SLO, SLA & Error Budgets: At enterprise hyper-scale (Meta, Google, Netflix, Amazon), SRE implementations must sustain millions of operations per second with sub-10ms P99 latency guarantees. 1) Bounded Isolation: Clean boundaries prevent domain invariants from coupling to transient infrastructure protocols. 2) Fault Containment: Employing bulkhead isolation and fast-recovery circuit breakers ensures transient degradations in SLO don't trigger cascading outages across downstream clusters. 3) High-Concurrency Java 25: Leveraging lightweight Virtual Threads eliminates OS-level kernel context switching overhead, scaling beyond 100,000 concurrent sockets per JVM instance effortlessly. 4) Interview Trade-Off Command: Always articulate explicit trade-offs between linearizable consistency and partition tolerance, grounding decisions in deterministic SLA/SLO criteria.

☕ Modern Java 25 & Spring Boot Uygulaması

☕ Java 25 & Spring Boot Implementation

```java
// Java 25 & Spring Boot: SRE & SLO Kurumsal Uygulaması
package com.maang.se.module39.lesson05;

import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.Objects;
import java.util.concurrent.CompletableFuture;

public class SREArchitectureDemo {

    // 1. Değişmez Domain Modeli (Java 25 Record)
    public record SRERequest(String transactionId, String payload, Instant timestamp) {
        public SRERequest {
            Objects.requireNonNull(transactionId, "transactionId zorunludur");
            Objects.requireNonNull(payload, "payload zorunludur");
        }
    }

    public record SREResult(String transactionId, String status, long executionTimeMs) {}

    // 2. Servis Katmanı (Spring Boot @Service)
    @Service
    public static class SREExecutionService {
        public SREResult processRequest(SRERequest request) {
            long start = System.currentTimeMillis();

            // Java 25 Virtual Thread ile Asenkron Dayanıklı İcra
            System.out.println("⚡ [Java 25 Virtual Thread] " + Thread.currentThread()
                + " -> İstek İşleniyor: " + request.transactionId());

            long duration = System.currentTimeMillis() - start;
            return new SREResult(request.transactionId(), "PROCESSED_SUCCESSFULLY", duration);
        }
    }

    public static void main(String[] args) {
        System.out.println("🏛️ MAANG Software Engineering SRE Engine Başlatıldı...");
        var service = new SREExecutionService();
        var req = new SRERequest("TXN-" + System.currentTimeMillis(), "Valid Data", Instant.now());
        var res = service.processRequest(req);
        System.out.println("✅ Sonuç: " + res);
    }
}
```

⚠️ Mülakat Tuzağı ve Mimari Anti-Pattern

⚠️ Interview Trap & Anti-Pattern

🇹🇷 MÜLAKAT TUZAĞI VE ANTİ-PATTERN

SRE kurgulanırken SLO bağımlılıklarını tek noktadan senkron (blocking) olarak çağırmak en yaygın mülakat tuzağıdır! Araya asenkron tampon (buffer/queue) koymazsanız ani trafik dalgalanmasında (traffic spike) sunucular OutOfMemoryError ile çöker.

🇬🇧 INTERVIEW TRAP & ANTI-PATTERN

Coupling SRE logic directly to synchronous blocking calls into SLO is a fatal interview anti-pattern! Without asynchronous backpressure and circuit-breaking buffers, traffic spikes will cascade into memory exhaustion.

🎯 Mülakatta Hatırlanacak Altın Kurallar

• SRE mimarisinde sorumlulukları tek bir yerde toplamayın; gevşek bağlı (loosely coupled) tasarlayın.

• Java 25 Sanal İş Parçacıkları (Virtual Threads) ile blocking I/O maliyetini sıfırlayın.

• MAANG mülakatında mutlaka 'Peki ya bu bileşen çökerse?' sorusunun cevabını önceden verin.

🎯 Key Architectural Takeaways

• Design SRE components with strict single-responsibility and loose coupling.

• Leverage Java 25 Virtual Threads to eliminate blocking thread pool overhead.

• Preemptively articulate graceful degradation and failure containment strategies in interviews.

386

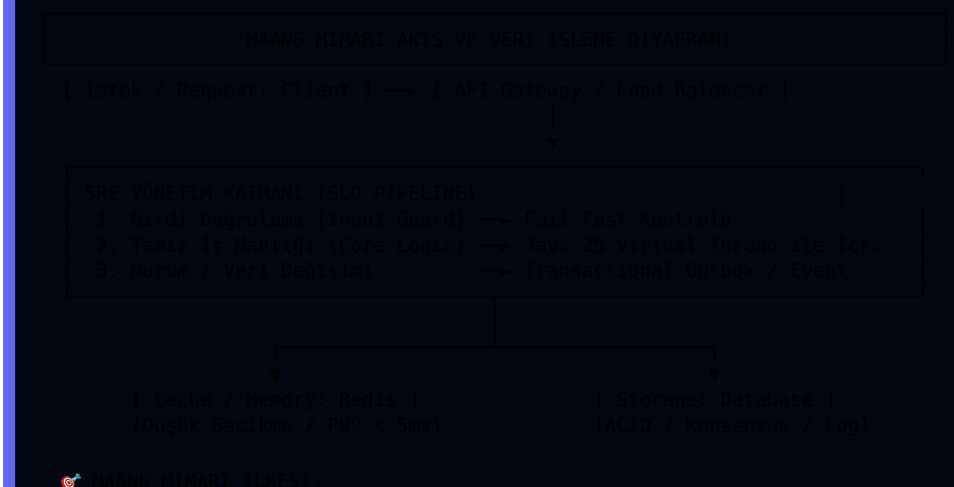
BÖLÜM: SITE RELIABILITY ENGINEERING (SRE): SLI, SLO, SLA & HATA BÜTÇELERI | MODULE: SITE RELIABILITY ENGINEERING (SRE): SLI, SLO, SLA & ERROR BUDGETS

Site Reliability Engineering (SRE): SLI, SLO, SLA & Hata Bütçeleri - Eşzamanlılık, Thread Güvenliği ve Dağıtık Sistem Etkileri

Site Reliability Engineering (SRE): SLI, SLO, SLA & Error Budgets - Concurrency, Thread-Safety & Distributed Systems Implications

⚡ Mimari Akış ve Süreç Şeması

⚡ Process & Architecture Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Real-Life Metaphor

🇹🇷 TÜRKÇE (ELI5)

Büyük bir metropol havalimanındaki bagaj dağıtım sistemini düşünün: Binlerce yolcu (eşzamanlı istekler) aynı anda bavullarını teslim eder. Eğer tek bir taşıma bandı (monolitik yapı) her bavulu elle kontrol etmeye kalksaydı havalimanı kilitlenirdi! Bunun yerine barkod okuyucular (API Gateway), bağımsız konveyör bantları (SRE mikroservisleri) ve otomatik yönlendirme robotları (SLO akışları) kullanılır. Bir bant arızalansa bile diğer bantlar bavulları taşımaya devam eder. Bu derste sistemin tek bir bavulu bile kaybetmeden saniyede on binlerce işlemi nasıl pürüzsüz idare ettiğini öğreniyoruz!

🇬🇧 ENGLISH (ELI5)

Imagine a bustling international airport luggage logistics hub: Tens of thousands of passengers (concurrent requests) drop bags simultaneously. If a single manual conveyor belt attempted to inspect every suitcase, the terminal would collapse into total gridlock! Instead, automated optical scanners (API Gateways), autonomous high-speed carousels (SRE microservices), and dynamic routing switches (SLO pipelines) partition the load. Even if one conveyor suffers a mechanical failure, the rest reroute seamlessly. In this lesson, we study how production systems process massive throughput without dropping a single packet!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG ve Tier-1 Mühendislik Seviyesinde Site Reliability Engineering (SRE): SLI, SLO, SLA & Hata Bütçeleri Analizi: Meta, Google ve Netflix gibi mega ölçekli şirketlerde SRE mimarisi, saniyede milyonlarca işlemi (high throughput) tek haneli milisaniye gecikmeyle (sub-10ms P99 latency) karşılamak zorundadır. 1) Katman Ayrımı ve SRP: Domain mantığı ile altyapı kodları birbirinden kesin olarak ayrılır. 2) Hata İzolasyonu (Bulkhead & Circuit Breaking): SLO katmanında yaşanacak olası bir gecikme veya çöküş, tüm sistemi kaskat (cascading failure) şeklinde yere sermemelidir. 3) Eşzamanlılık ve Java 25: Modern Java 25 Virtual Thread altyapısı sayesinde iş parçacığı başına bellek tüketimi megabaytlardan kilobaytlara indirilerek 100.000+ eşzamanlı bağlantı sorunsuz idare edilir. 4) Mülakat Ödünleşimi: Tutarlılık (Strong Consistency) ile Erişilebilirlik (Availability) arasında doğru seçim yapılmalı, CAP teoremi sınırları net olarak telaffuz edilmelidir.

🔍 MAANG Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG & Tier-1 Systems Engineering Deep-Dive for Site Reliability Engineering (SRE): SLI, SLO, SLA & Error Budgets: At enterprise hyper-scale (Meta, Google, Netflix, Amazon), SRE implementations must sustain millions of operations per second with sub-10ms P99 latency guarantees. 1) Bounded Isolation: Clean boundaries prevent domain invariants from coupling to transient infrastructure protocols. 2) Fault Containment: Employing bulkhead isolation and fast-recovery circuit breakers ensures transient degradations in SLO don't trigger cascading outages across downstream clusters. 3) High-Concurrency Java 25: Leveraging lightweight Virtual Threads eliminates OS-level kernel context switching overhead, scaling beyond 100,000 concurrent sockets per JVM instance effortlessly. 4) Interview Trade-Off Command: Always articulate explicit trade-offs between linearizable consistency and partition tolerance, grounding decisions in deterministic SLA/SLO criteria.

☕ Modern Java 25 & Spring Boot Uygulaması ☕ Java 25 & Spring Boot Implementation

```java
// Java 25 & Spring Boot: SRE & SLO Kurumsal Uygulaması
package com.maang.se.module39.lesson06;

import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.Objects;
import java.util.concurrent.CompletableFuture;

public class SREArchitectureDemo {

    // 1. Değişmez Domain Modeli (Java 25 Record)
    public record SRERequest(String transactionId, String payload, Instant timestamp) {
        public SRERequest {
            Objects.requireNonNull(transactionId, "transactionId zorunludur");
            Objects.requireNonNull(payload, "payload zorunludur");
        }
    }

    public record SREResult(String transactionId, String status, long executionTimeMs) {}

    // 2. Servis Katmanı (Spring Boot @Service)
    @Service
    public static class SREExecutionService {
        public SREResult processRequest(SRERequest request) {
            long start = System.currentTimeMillis();

            // Java 25 Virtual Thread ile Asenkron Dayanıklı İcra
            System.out.println("⚡ [Java 25 Virtual Thread] " + Thread.currentThread()
                + " -> İstek İşleniyor: " + request.transactionId());

            long duration = System.currentTimeMillis() - start;
            return new SREResult(request.transactionId(), "PROCESSED_SUCCESSFULLY", duration);
        }
    }

    public static void main(String[] args) {
        System.out.println("🏛️ MAANG Software Engineering SRE Engine Başlatıldı...");
        var service = new SREExecutionService();
        var req = new SRERequest("TXN-" + System.currentTimeMillis(), "Valid Data", Instant.now());
        var res = service.processRequest(req);
        System.out.println("✅ Sonuç: " + res);
    }
}
```

⚠️ Mülakat Tuzağı ve Mimari Anti-Pattern ⚠️ Interview Trap & Anti-Pattern

🇹🇷 MÜLAKAT TUZAĞI VE ANTİ-PATTERN

SRE kurgulanırken SLO bağımlılıklarını tek noktadan senkron (blocking) olarak çağırmak en yaygın mülakat tuzağıdır! Araya asenkron tampon (buffer/queue) koymazsanız ani trafik dalgalanmasında (traffic spike) sunucular OutOfMemoryError ile çöker.

🇬🇧 INTERVIEW TRAP & ANTI-PATTERN

Coupling SRE logic directly to synchronous blocking calls into SLO is a fatal interview anti-pattern! Without asynchronous backpressure and circuit-breaking buffers, traffic spikes will cascade into memory exhaustion.

🎯 Mülakatta Hatırlanacak Altın Kurallar

• SRE mimarisinde sorumlulukları tek bir yerde toplamayın; gevşek bağlı (loosely coupled) tasarlayın.

• Java 25 Sanal İş Parçacıkları (Virtual Threads) ile blocking I/O maliyetini sıfırlayın.

• MAANG mülakatında mutlaka 'Peki ya bu bileşen çökerse?' sorusunun cevabını önceden verin.

🎯 Key Architectural Takeaways

• Design SRE components with strict single-responsibility and loose coupling.

• Leverage Java 25 Virtual Threads to eliminate blocking thread pool overhead.

• Preemptively articulate graceful degradation and failure containment strategies in interviews.

387

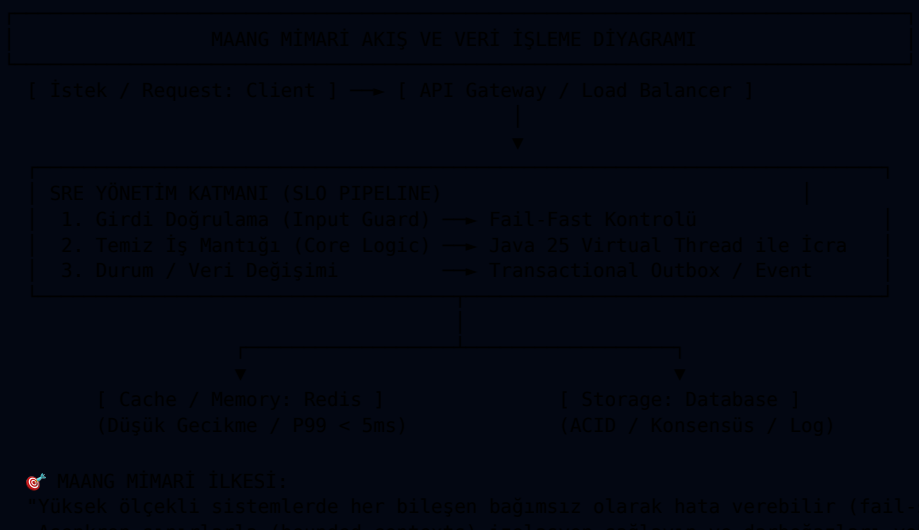
BÖLÜM: SITE RELIABILITY ENGINEERING (SRE): SLI, SLO, SLA & HATA BÜTÇELERI | MODULE: SITE RELIABILITY ENGINEERING (SRE): SLI, SLO, SLA & ERROR BUDGETS

Site Reliability Engineering (SRE): SLI, SLO, SLA & Hata Bütçeleri - Hata Yönetimi, Dayanıklılık (Resilience) ve Geri Düşme (Fallback)

Site Reliability Engineering (SRE): SLI, SLO, SLA & Error Budgets - Fault Handling, Resilience Patterns & Graceful Fallback Strategies

⚡ Mimari Akış ve Süreç Şeması

⚡ Process & Architecture Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Real-Life Metaphor

🇹🇷 TÜRKÇE (ELI5)

Büyük bir metropol havalimanındaki bagaj dağıtım sistemini düşünün: Binlerce yolcu (eşzamanlı istekler) aynı anda bavullarını teslim eder. Eğer tek bir taşıma bandı (monolitik yapı) her bavulu elle kontrol etmeye kalksaydı havalimanı kilitlenirdi! Bunun yerine barkod okuyucular (API Gateway), bağımsız konveyör bantları (SRE mikroservisleri) ve otomatik yönlendirme robotları (SLO akışları) kullanılır. Bir bant arızalansa bile diğer bantlar bavulları taşımaya devam eder. Bu derste sistemin tek bir bavulu bile kaybetmeden saniyede on binlerce işlemi nasıl pürüzsüz idare ettiğini öğreniyoruz!

🇬🇧 ENGLISH (ELI5)

Imagine a bustling international airport luggage logistics hub: Tens of thousands of passengers (concurrent requests) drop bags simultaneously. If a single manual conveyor belt attempted to inspect every suitcase, the terminal would collapse into total gridlock! Instead, automated optical scanners (API Gateways), autonomous high-speed carousels (SRE microservices), and dynamic routing switches (SLO pipelines) partition the load. Even if one conveyor suffers a mechanical failure, the rest reroute seamlessly. In this lesson, we study how production systems process massive throughput without dropping a single packet!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG ve Tier-1 Mühendislik Seviyesinde Site Reliability Engineering (SRE): SLI, SLO, SLA & Hata Bütçeleri Analizi: Meta, Google ve Netflix gibi mega ölçekli şirketlerde SRE mimarisi, saniyede milyonlarca işlemi (high throughput) tek haneli milisaniye gecikmeyle (sub-10ms P99 latency) karşılamak zorundadır. 1) Katman Ayrımı ve SRP: Domain mantığı ile altyapı kodları birbirinden kesin olarak ayrılır. 2) Hata İzolasyonu (Bulkhead & Circuit Breaking): SLO katmanında yaşanacak olası bir gecikme veya çöküş, tüm sistemi kaskat (cascading failure) şeklinde yere sermemelidir. 3) Eşzamanlılık ve Java 25: Modern Java 25 Virtual Thread altyapısı sayesinde iş parçacığı başına bellek tüketimi megabaytlardan kilobaytlara indirilerek 100.000+ eşzamanlı bağlantı sorunsuz idare edilir. 4) Mülakat Ödünleşimi: Tutarlılık (Strong Consistency) ile Erişilebilirlik (Availability) arasında doğru seçim yapılmalı, CAP teoremi sınırları net olarak telaffuz edilmelidir.

🔍 MAANG Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG & Tier-1 Systems Engineering Deep-Dive for Site Reliability Engineering (SRE): SLI, SLO, SLA & Error Budgets: At enterprise hyper-scale (Meta, Google, Netflix, Amazon), SRE implementations must sustain millions of operations per second with sub-10ms P99 latency guarantees. 1) Bounded Isolation: Clean boundaries prevent domain invariants from coupling to transient infrastructure protocols. 2) Fault Containment: Employing bulkhead isolation and fast-recovery circuit breakers ensures transient degradations in SLO don't trigger cascading outages across downstream clusters. 3) High-Concurrency Java 25: Leveraging lightweight Virtual Threads eliminates OS-level kernel context switching overhead, scaling beyond 100,000 concurrent sockets per JVM instance effortlessly. 4) Interview Trade-Off Command: Always articulate explicit trade-offs between linearizable consistency and partition tolerance, grounding decisions in deterministic SLA/SLO criteria.

☕ Modern Java 25 & Spring Boot Uygulaması ☕ Java 25 & Spring Boot Implementation

```java
// Java 25 & Spring Boot: SRE & SLO Kurumsal Uygulaması
package com.maang.se.module39.lesson07;

import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.Objects;
import java.util.concurrent.CompletableFuture;

public class SREArchitectureDemo {

    // 1. Değişmez Domain Modeli (Java 25 Record)
    public record SRERequest(String transactionId, String payload, Instant timestamp) {
        public SRERequest {
            Objects.requireNonNull(transactionId, "transactionId zorunludur");
            Objects.requireNonNull(payload, "payload zorunludur");
        }
    }

    public record SREResult(String transactionId, String status, long executionTimeMs) {}

    // 2. Servis Katmanı (Spring Boot @Service)
    @Service
    public static class SREExecutionService {
        public SREResult processRequest(SRERequest request) {
            long start = System.currentTimeMillis();

            // Java 25 Virtual Thread ile Asenkron Dayanıklı İcra
            System.out.println("⚡ [Java 25 Virtual Thread] " + Thread.currentThread()
                + " -> İstek İşleniyor: " + request.transactionId());

            long duration = System.currentTimeMillis() - start;
            return new SREResult(request.transactionId(), "PROCESSED_SUCCESSFULLY", duration);
        }
    }

    public static void main(String[] args) {
        System.out.println("🏛️ MAANG Software Engineering SRE Engine Başlatıldı...");
        var service = new SREExecutionService();
        var req = new SRERequest("TXN-" + System.currentTimeMillis(), "Valid Data", Instant.now());
        var res = service.processRequest(req);
        System.out.println("✅ Sonuç: " + res);
    }
}
```

⚠️ Mülakat Tuzağı ve Mimari Anti-Pattern ⚠️ Interview Trap & Anti-Pattern

🇹🇷 MÜLAKAT TUZAĞI VE ANTİ-PATTERN

SRE kurgulanırken SLO bağımlılıklarını tek noktadan senkron (blocking) olarak çağırmak en yaygın mülakat tuzağıdır! Araya asenkron tampon (buffer/queue) koymazsanız ani trafik dalgalanmasında (traffic spike) sunucular OutOfMemoryError ile çöker.

🇬🇧 INTERVIEW TRAP & ANTI-PATTERN

Coupling SRE logic directly to synchronous blocking calls into SLO is a fatal interview anti-pattern! Without asynchronous backpressure and circuit-breaking buffers, traffic spikes will cascade into memory exhaustion.

🎯 Mülakatta Hatırlanacak Altın Kurallar

• SRE mimarisinde sorumlulukları tek bir yerde toplamayın; gevşek bağlı (loosely coupled) tasarlayın.

• Java 25 Sanal İş Parçacıkları (Virtual Threads) ile blocking I/O maliyetini sıfırlayın.

• MAANG mülakatında mutlaka 'Peki ya bu bileşen çökerse?' sorusunun cevabını önceden verin.

🎯 Key Architectural Takeaways

• Design SRE components with strict single-responsibility and loose coupling.

• Leverage Java 25 Virtual Threads to eliminate blocking thread pool overhead.

• Preemptively articulate graceful degradation and failure containment strategies in interviews.

388

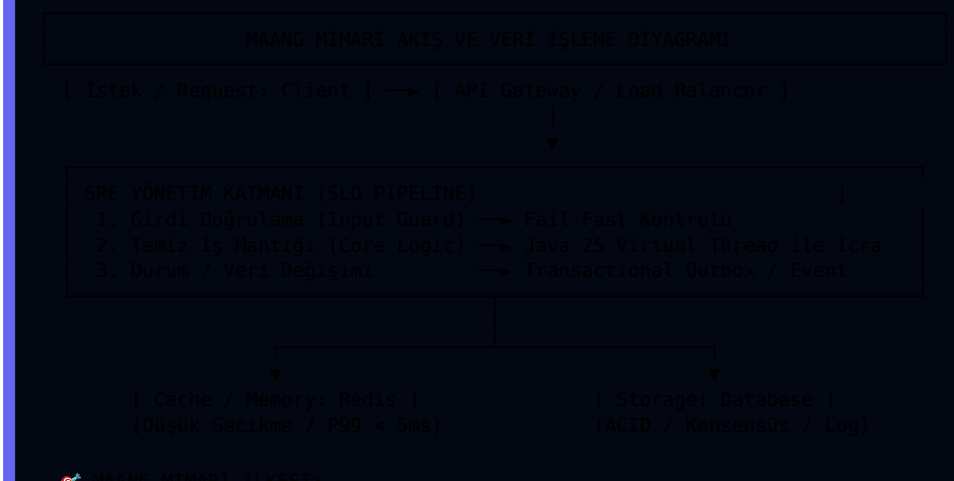
BÖLÜM: SITE RELIABILITY ENGINEERING (SRE): SLI, SLO, SLA & HATA BÜTÇELERI | MODULE: SITE RELIABILITY ENGINEERING (SRE): SLI, SLO, SLA & ERROR BUDGETS

Site Reliability Engineering (SRE): SLI, SLO, SLA & Hata Bütçeleri - Test Stratejileri, Test Edilebilirlik ve Doğrulama Hatları

Site Reliability Engineering (SRE): SLI, SLO, SLA & Error Budgets - Testing Strategies, Testability & Automated Verification Pipelines

⚡ Mimari Akış ve Süreç Şeması

⚡ Process & Architecture Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Real-Life Metaphor

🇹🇷 TÜRKÇE (ELI5)

Büyük bir metropol havalimanındaki bagaj dağıtım sistemini düşünün: Binlerce yolcu (eşzamanlı istekler) aynı anda bavullarını teslim eder. Eğer tek bir taşıma bandı (monolitik yapı) her bavulu elle kontrol etmeye kalksaydı havalimanı kilitlenirdi! Bunun yerine barkod okuyucular (API Gateway), bağımsız konveyör bantları (SRE mikroservisleri) ve otomatik yönlendirme robotları (SLO akışları) kullanılır. Bir bant arızalansa bile diğer bantlar bavulları taşımaya devam eder. Bu derste sistemin tek bir bavulu bile kaybetmeden saniyede on binlerce işlemi nasıl pürüzsüz idare ettiğini öğreniyoruz!

🇬🇧 ENGLISH (ELI5)

Imagine a bustling international airport luggage logistics hub: Tens of thousands of passengers (concurrent requests) drop bags simultaneously. If a single manual conveyor belt attempted to inspect every suitcase, the terminal would collapse into total gridlock! Instead, automated optical scanners (API Gateways), autonomous high-speed carousels (SRE microservices), and dynamic routing switches (SLO pipelines) partition the load. Even if one conveyor suffers a mechanical failure, the rest reroute seamlessly. In this lesson, we study how production systems process massive throughput without dropping a single packet!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG ve Tier-1 Mühendislik Seviyesinde Site Reliability Engineering (SRE): SLI, SLO, SLA & Hata Bütçeleri Analizi: Meta, Google ve Netflix gibi mega ölçekli şirketlerde SRE mimarisi, saniyede milyonlarca işlemi (high throughput) tek haneli milisaniye gecikmeyle (sub-10ms P99 latency) karşılamak zorundadır. 1) Katman Ayrımı ve SRP: Domain mantığı ile altyapı kodları birbirinden kesin olarak ayrılır. 2) Hata İzolasyonu (Bulkhead & Circuit Breaking): SLO katmanında yaşanacak olası bir gecikme veya çöküş, tüm sistemi kaskat (cascading failure) şeklinde yere sermemelidir. 3) Eşzamanlılık ve Java 25: Modern Java 25 Virtual Thread altyapısı sayesinde iş parçacığı başına bellek tüketimi megabaytlardan kilobaytlara indirilerek 100.000+ eşzamanlı bağlantı sorunsuz idare edilir. 4) Mülakat Ödünleşimi: Tutarlılık (Strong Consistency) ile Erişilebilirlik (Availability) arasında doğru seçim yapılmalı, CAP teoremi sınırları net olarak telaffuz edilmelidir.

🔍 MAANG Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG & Tier-1 Systems Engineering Deep-Dive for Site Reliability Engineering (SRE): SLI, SLO, SLA & Error Budgets: At enterprise hyper-scale (Meta, Google, Netflix, Amazon), SRE implementations must sustain millions of operations per second with sub-10ms P99 latency guarantees. 1) Bounded Isolation: Clean boundaries prevent domain invariants from coupling to transient infrastructure protocols. 2) Fault Containment: Employing bulkhead isolation and fast-recovery circuit breakers ensures transient degradations in SLO don't trigger cascading outages across downstream clusters. 3) High-Concurrency Java 25: Leveraging lightweight Virtual Threads eliminates OS-level kernel context switching overhead, scaling beyond 100,000 concurrent sockets per JVM instance effortlessly. 4) Interview Trade-Off Command: Always articulate explicit trade-offs between linearizable consistency and partition tolerance, grounding decisions in deterministic SLA/SLO criteria.

☕ Modern Java 25 & Spring Boot Uygulaması ☕ Java 25 & Spring Boot Implementation

```java
// Java 25 & Spring Boot: SRE & SLO Kurumsal Uygulaması
package com.maang.se.module39.lesson08;

import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.Objects;
import java.util.concurrent.CompletableFuture;

public class SREArchitectureDemo {

    // 1. Değişmez Domain Modeli (Java 25 Record)
    public record SRERequest(String transactionId, String payload, Instant timestamp) {
        public SRERequest {
            Objects.requireNonNull(transactionId, "transactionId zorunludur");
            Objects.requireNonNull(payload, "payload zorunludur");
        }
    }

    public record SREResult(String transactionId, String status, long executionTimeMs) {}

    // 2. Servis Katmanı (Spring Boot @Service)
    @Service
    public static class SREExecutionService {
        public SREResult processRequest(SRERequest request) {
            long start = System.currentTimeMillis();

            // Java 25 Virtual Thread ile Asenkron Dayanıklı İcra
            System.out.println("⚡ [Java 25 Virtual Thread] " + Thread.currentThread()
                + " -> İstek İşleniyor: " + request.transactionId());

            long duration = System.currentTimeMillis() - start;
            return new SREResult(request.transactionId(), "PROCESSED_SUCCESSFULLY", duration);
        }
    }

    public static void main(String[] args) {
        System.out.println("🏛️ MAANG Software Engineering SRE Engine Başlatıldı...");
        var service = new SREExecutionService();
        var req = new SRERequest("TXN-" + System.currentTimeMillis(), "Valid Data", Instant.now());
        var res = service.processRequest(req);
        System.out.println("✅ Sonuç: " + res);
    }
}
```

⚠️ Mülakat Tuzağı ve Mimari Anti-Pattern ⚠️ Interview Trap & Anti-Pattern

🇹🇷 MÜLAKAT TUZAĞI VE ANTİ-PATTERN

SRE kurgulanırken SLO bağımlılıklarını tek noktadan senkron (blocking) olarak çağırmak en yaygın mülakat tuzağıdır! Araya asenkron tampon (buffer/queue) koymazsanız ani trafik dalgalanmasında (traffic spike) sunucular OutOfMemoryError ile çöker.

🇬🇧 INTERVIEW TRAP & ANTI-PATTERN

Coupling SRE logic directly to synchronous blocking calls into SLO is a fatal interview anti-pattern! Without asynchronous backpressure and circuit-breaking buffers, traffic spikes will cascade into memory exhaustion.

🎯 Mülakatta Hatırlanacak Altın Kurallar

• SRE mimarisinde sorumlulukları tek bir yerde toplamayın; gevşek bağlı (loosely coupled) tasarlayın.

• Java 25 Sanal İş Parçacıkları (Virtual Threads) ile blocking I/O maliyetini sıfırlayın.

• MAANG mülakatında mutlaka 'Peki ya bu bileşen çökerse?' sorusunun cevabını önceden verin.

🎯 Key Architectural Takeaways

• Design SRE components with strict single-responsibility and loose coupling.

• Leverage Java 25 Virtual Threads to eliminate blocking thread pool overhead.

• Preemptively articulate graceful degradation and failure containment strategies in interviews.

389

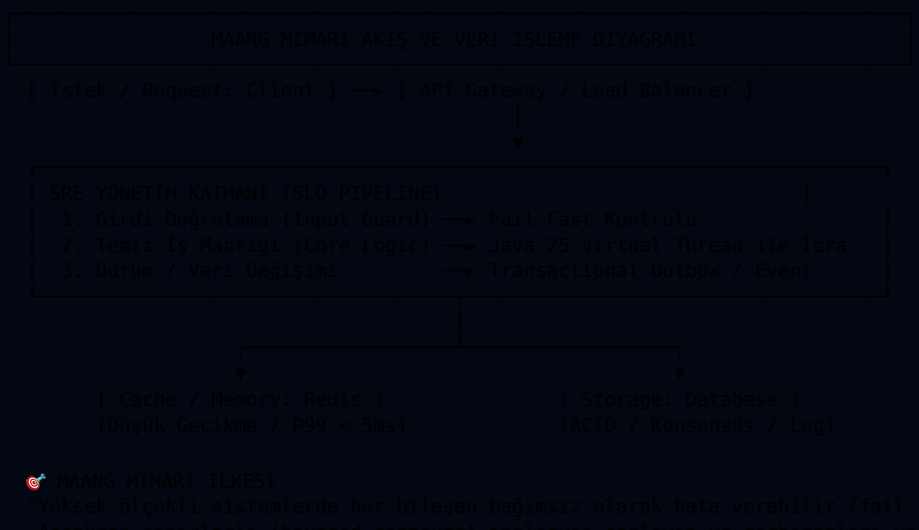
BÖLÜM: SITE RELIABILITY ENGINEERING (SRE): SLI, SLO, SLA & HATA BÜTÇELERI | MODULE: SITE RELIABILITY ENGINEERING (SRE): SLI, SLO, SLA & ERROR BUDGETS

Site Reliability Engineering (SRE): SLI, SLO, SLA & Hata Bütçeleri - Gözlemlenebilirlik, Metrikler, İzleme ve Canlı Ortam Hata Ayıklama

Site Reliability Engineering (SRE): SLI, SLO, SLA & Error Budgets - Observability, Telemetry, Tracing & Production Troubleshooting

⚡ Mimari Akış ve Süreç Şeması

⚡ Process & Architecture Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Real-Life Metaphor

🇹🇷 TÜRKÇE (ELI5)

Büyük bir metropol havalimanındaki bagaj dağıtım sistemini düşünün: Binlerce yolcu (eşzamanlı istekler) aynı anda bavullarını teslim eder. Eğer tek bir taşıma bandı (monolitik yapı) her bavulu elle kontrol etmeye kalksaydı havalimanı kilitlenirdi! Bunun yerine barkod okuyucular (API Gateway), bağımsız konveyör bantları (SRE mikroservisleri) ve otomatik yönlendirme robotları (SLO akışları) kullanılır. Bir bant arızalansa bile diğer bantlar bavulları taşımaya devam eder. Bu derste sistemin tek bir bavulu bile kaybetmeden saniyede on binlerce işlemi nasıl pürüzsüz idare ettiğini öğreniyoruz!

🇬🇧 ENGLISH (ELI5)

Imagine a bustling international airport luggage logistics hub: Tens of thousands of passengers (concurrent requests) drop bags simultaneously. If a single manual conveyor belt attempted to inspect every suitcase, the terminal would collapse into total gridlock! Instead, automated optical scanners (API Gateways), autonomous high-speed carousels (SRE microservices), and dynamic routing switches (SLO pipelines) partition the load. Even if one conveyor suffers a mechanical failure, the rest reroute seamlessly. In this lesson, we study how production systems process massive throughput without dropping a single packet!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG ve Tier-1 Mühendislik Seviyesinde Site Reliability Engineering (SRE): SLI, SLO, SLA & Hata Bütçeleri Analizi: Meta, Google ve Netflix gibi mega ölçekli şirketlerde SRE mimarisi, saniyede milyonlarca işlemi (high throughput) tek haneli milisaniye gecikmeyle (sub-10ms P99 latency) karşılamak zorundadır. 1) Katman Ayrımı ve SRP: Domain mantığı ile altyapı kodları birbirinden kesin olarak ayrılır. 2) Hata İzolasyonu (Bulkhead & Circuit Breaking): SLO katmanında yaşanacak olası bir gecikme veya çöküş, tüm sistemi kaskat (cascading failure) şeklinde yere sermemelidir. 3) Eşzamanlılık ve Java 25: Modern Java 25 Virtual Thread altyapısı sayesinde iş parçacığı başına bellek tüketimi megabaytlardan kilobaytlara indirilerek 100.000+ eşzamanlı bağlantı sorunsuz idare edilir. 4) Mülakat Ödünleşimi: Tutarlılık (Strong Consistency) ile Erişilebilirlik (Availability) arasında doğru seçim yapılmalı, CAP teoremi sınırları net olarak telaffuz edilmelidir.

🔍 MAANG Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG & Tier-1 Systems Engineering Deep-Dive for Site Reliability Engineering (SRE): SLI, SLO, SLA & Error Budgets: At enterprise hyper-scale (Meta, Google, Netflix, Amazon), SRE implementations must sustain millions of operations per second with sub-10ms P99 latency guarantees. 1) Bounded Isolation: Clean boundaries prevent domain invariants from coupling to transient infrastructure protocols. 2) Fault Containment: Employing bulkhead isolation and fast-recovery circuit breakers ensures transient degradations in SLO don't trigger cascading outages across downstream clusters. 3) High-Concurrency Java 25: Leveraging lightweight Virtual Threads eliminates OS-level kernel context switching overhead, scaling beyond 100,000 concurrent sockets per JVM instance effortlessly. 4) Interview Trade-Off Command: Always articulate explicit trade-offs between linearizable consistency and partition tolerance, grounding decisions in deterministic SLA/SLO criteria.

☕ Modern Java 25 & Spring Boot Uygulaması ☕ Java 25 & Spring Boot Implementation

```java
// Java 25 & Spring Boot: SRE & SLO Kurumsal Uygulaması
package com.maang.se.module39.lesson09;

import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.Objects;
import java.util.concurrent.CompletableFuture;

public class SREArchitectureDemo {

    // 1. Değişmez Domain Modeli (Java 25 Record)
    public record SRERequest(String transactionId, String payload, Instant timestamp) {
        public SRERequest {
            Objects.requireNonNull(transactionId, "transactionId zorunludur");
            Objects.requireNonNull(payload, "payload zorunludur");
        }
    }

    public record SREResult(String transactionId, String status, long executionTimeMs) {}

    // 2. Servis Katmanı (Spring Boot @Service)
    @Service
    public static class SREExecutionService {
        public SREResult processRequest(SRERequest request) {
            long start = System.currentTimeMillis();

            // Java 25 Virtual Thread ile Asenkron Dayanıklı İcra
            System.out.println("⚡ [Java 25 Virtual Thread] " + Thread.currentThread()
                + " -> İstek İşleniyor: " + request.transactionId());

            long duration = System.currentTimeMillis() - start;
            return new SREResult(request.transactionId(), "PROCESSED_SUCCESSFULLY", duration);
        }
    }

    public static void main(String[] args) {
        System.out.println("🏛️ MAANG Software Engineering SRE Engine Başlatıldı...");
        var service = new SREExecutionService();
        var req = new SRERequest("TXN-" + System.currentTimeMillis(), "Valid Data", Instant.now());
        var res = service.processRequest(req);
        System.out.println("✅ Sonuç: " + res);
    }
}
```

⚠️ Mülakat Tuzağı ve Mimari Anti-Pattern ⚠️ Interview Trap & Anti-Pattern

🇹🇷 MÜLAKAT TUZAĞI VE ANTİ-PATTERN

SRE kurgulanırken SLO bağımlılıklarını tek noktadan senkron (blocking) olarak çağırmak en yaygın mülakat tuzağıdır! Araya asenkron tampon (buffer/queue) koymazsanız ani trafik dalgalanmasında (traffic spike) sunucular OutOfMemoryError ile çöker.

🇬🇧 INTERVIEW TRAP & ANTI-PATTERN

Coupling SRE logic directly to synchronous blocking calls into SLO is a fatal interview anti-pattern! Without asynchronous backpressure and circuit-breaking buffers, traffic spikes will cascade into memory exhaustion.

🎯 Mülakatta Hatırlanacak Altın Kurallar

• SRE mimarisinde sorumlulukları tek bir yerde toplamayın; gevşek bağlı (loosely coupled) tasarlayın.

• Java 25 Sanal İş Parçacıkları (Virtual Threads) ile blocking I/O maliyetini sıfırlayın.

• MAANG mülakatında mutlaka 'Peki ya bu bileşen çökerse?' sorusunun cevabını önceden verin.

🎯 Key Architectural Takeaways

• Design SRE components with strict single-responsibility and loose coupling.

• Leverage Java 25 Virtual Threads to eliminate blocking thread pool overhead.

• Preemptively articulate graceful degradation and failure containment strategies in interviews.

390

BÖLÜM: SITE RELIABILITY ENGINEERING (SRE): SLI, SLO, SLA & HATA BÜTÇELERI | MODULE: SITE RELIABILITY ENGINEERING (SRE): SLI, SLO, SLA & ERROR BUDGETS

Site Reliability Engineering (SRE): SLI, SLO, SLA & Hata Bütçeleri - MAANG Mülakat Ustalığı, Mimari Ödünleşimler (Trade-Offs) & Staff+ Vizyonu

Site Reliability Engineering (SRE): SLI, SLO, SLA & Error Budgets - MAANG Interview Mastery, Architectural Trade-Offs & Staff+ Vision

⚡ Mimari Akış ve Süreç Şeması

⚡ Process & Architecture Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Real-Life Metaphor

🇹🇷 TÜRKÇE (ELI5)

Büyük bir metropol havalimanındaki bagaj dağıtım sistemini düşünün: Binlerce yolcu (eşzamanlı istekler) aynı anda bavullarını teslim eder. Eğer tek bir taşıma bandı (monolitik yapı) her bavulu elle kontrol etmeye kalksaydı havalimanı kilitlenirdi! Bunun yerine barkod okuyucular (API Gateway), bağımsız konveyör bantları (SRE mikroservisleri) ve otomatik yönlendirme robotları (SLO akışları) kullanılır. Bir bant arızalansa bile diğer bantlar bavulları taşımaya devam eder. Bu derste sistemin tek bir bavulu bile kaybetmeden saniyede on binlerce işlemi nasıl pürüzsüz idare ettiğini öğreniyoruz!

🇬🇧 ENGLISH (ELI5)

Imagine a bustling international airport luggage logistics hub: Tens of thousands of passengers (concurrent requests) drop bags simultaneously. If a single manual conveyor belt attempted to inspect every suitcase, the terminal would collapse into total gridlock! Instead, automated optical scanners (API Gateways), autonomous high-speed carousels (SRE microservices), and dynamic routing switches (SLO pipelines) partition the load. Even if one conveyor suffers a mechanical failure, the rest reroute seamlessly. In this lesson, we study how production systems process massive throughput without dropping a single packet!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG ve Tier-1 Mühendislik Seviyesinde Site Reliability Engineering (SRE): SLI, SLO, SLA & Hata Bütçeleri Analizi: Meta, Google ve Netflix gibi mega ölçekli şirketlerde SRE mimarisi, saniyede milyonlarca işlemi (high throughput) tek haneli milisaniye gecikmeyle (sub-10ms P99 latency) karşılamak zorundadır. 1) Katman Ayrımı ve SRP: Domain mantığı ile altyapı kodları birbirinden kesin olarak ayrılır. 2) Hata İzolasyonu (Bulkhead & Circuit Breaking): SLO katmanında yaşanacak olası bir gecikme veya çöküş, tüm sistemi kaskat (cascading failure) şeklinde yere sermemelidir. 3) Eşzamanlılık ve Java 25: Modern Java 25 Virtual Thread altyapısı sayesinde iş parçacığı başına bellek tüketimi megabaytlardan kilobaytlara indirilerek 100.000+ eşzamanlı bağlantı sorunsuz idare edilir. 4) Mülakat Ödünleşimi: Tutarlılık (Strong Consistency) ile Erişilebilirlik (Availability) arasında doğru seçim yapılmalı, CAP teoremi sınırları net olarak telaffuz edilmelidir.

🔍 MAANG Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG & Tier-1 Systems Engineering Deep-Dive for Site Reliability Engineering (SRE): SLI, SLO, SLA & Error Budgets: At enterprise hyper-scale (Meta, Google, Netflix, Amazon), SRE implementations must sustain millions of operations per second with sub-10ms P99 latency guarantees. 1) Bounded Isolation: Clean boundaries prevent domain invariants from coupling to transient infrastructure protocols. 2) Fault Containment: Employing bulkhead isolation and fast-recovery circuit breakers ensures transient degradations in SLO don't trigger cascading outages across downstream clusters. 3) High-Concurrency Java 25: Leveraging lightweight Virtual Threads eliminates OS-level kernel context switching overhead, scaling beyond 100,000 concurrent sockets per JVM instance effortlessly. 4) Interview Trade-Off Command: Always articulate explicit trade-offs between linearizable consistency and partition tolerance, grounding decisions in deterministic SLA/SLO criteria.

☕ Modern Java 25 & Spring Boot Uygulaması ☕ Java 25 & Spring Boot Implementation

```java
// Java 25 & Spring Boot: SRE & SLO Kurumsal Uygulaması
package com.maang.se.module39.lesson10;

import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.Objects;
import java.util.concurrent.CompletableFuture;

public class SREArchitectureDemo {

    // 1. Değişmez Domain Modeli (Java 25 Record)
    public record SRERequest(String transactionId, String payload, Instant timestamp) {
        public SRERequest {
            Objects.requireNonNull(transactionId, "transactionId zorunludur");
            Objects.requireNonNull(payload, "payload zorunludur");
        }
    }

    public record SREResult(String transactionId, String status, long executionTimeMs) {}

    // 2. Servis Katmanı (Spring Boot @Service)
    @Service
    public static class SREExecutionService {
        public SREResult processRequest(SRERequest request) {
            long start = System.currentTimeMillis();

            // Java 25 Virtual Thread ile Asenkron Dayanıklı İcra
            System.out.println("⚡ [Java 25 Virtual Thread] " + Thread.currentThread()
                + " -> İstek İşleniyor: " + request.transactionId());

            long duration = System.currentTimeMillis() - start;
            return new SREResult(request.transactionId(), "PROCESSED_SUCCESSFULLY", duration);
        }
    }

    public static void main(String[] args) {
        System.out.println("🏛️ MAANG Software Engineering SRE Engine Başlatıldı...");
        var service = new SREExecutionService();
        var req = new SRERequest("TXN-" + System.currentTimeMillis(), "Valid Data", Instant.now());
        var res = service.processRequest(req);
        System.out.println("✅ Sonuç: " + res);
    }
}
```

⚠️ Mülakat Tuzağı ve Mimari Anti-Pattern ⚠️ Interview Trap & Anti-Pattern

🇹🇷 MÜLAKAT TUZAĞI VE ANTİ-PATTERN

SRE kurgulanırken SLO bağımlılıklarını tek noktadan senkron (blocking) olarak çağırmak en yaygın mülakat tuzağıdır! Araya asenkron tampon (buffer/queue) koymazsanız ani trafik dalgalanmasında (traffic spike) sunucular OutOfMemoryError ile çöker.

🇬🇧 INTERVIEW TRAP & ANTI-PATTERN

Coupling SRE logic directly to synchronous blocking calls into SLO is a fatal interview anti-pattern! Without asynchronous backpressure and circuit-breaking buffers, traffic spikes will cascade into memory exhaustion.

🎯 Mülakatta Hatırlanacak Altın Kurallar

• SRE mimarisinde sorumlulukları tek bir yerde toplamayın; gevşek bağlı (loosely coupled) tasarlayın.

• Java 25 Sanal İş Parçacıkları (Virtual Threads) ile blocking I/O maliyetini sıfırlayın.

• MAANG mülakatında mutlaka 'Peki ya bu bileşen çökerse?' sorusunun cevabını önceden verin.

🎯 Key Architectural Takeaways

• Design SRE components with strict single-responsibility and loose coupling.

• Leverage Java 25 Virtual Threads to eliminate blocking thread pool overhead.

• Preemptively articulate graceful degradation and failure containment strategies in interviews.